

Jairs

1 September 2026

A Jai-inspired systems language in Rust. Incremental salsa front end, lossless CST, typed SSA mid-end, and **three** execution engines — a bytecode VM, Cranelift and LLVM — held byte-identical by a differential harness.

7/7 gates green 1033 tests ADR-0149 latest W8 DONE · 8 sub-waves W6 + W7 OPEN

TESTS

CORPUS

ADRS

DIAGNOSTICS

EDITOR CHECKS

1033

237

149

118

166

workspace; 1034 with LLVM in

jr files, all three engines

0001 to 0149, immutable

codes, E0286 next free

Neovim, verified not gated

The vertical slice, and what it costs to be honest about it

EXIT CRITERIA

DONE hello.jr runs in the VM
DONE compiles to a native binary
DONE a **third** engine agrees (LLVM)
DONE rustc-grade diagnostics
DONE formatter round-trips the corpus
DONE editor integration (Neovim)
OPEN verified Linux x86-64 CI run

The last one needs a push. Configured, never run, so it is a decision rather than a technical gap.

HOW CORRECTNESS IS ESTABLISHED

All three engines run every corpus program and must agree, on output and on trap wording, from one shared formatter. The LLVM back end agreed with the VM on all 114 executable programs on its **first** run — because both native engines read the same MIR and ask jr-pool the same layout questions, which is what ADR-0018 §2 centralising layout bought.

Probe the premise by writing the thing. Eight times now. The sharpest was #simd: Cranelift's type **constructor** happily makes a 256-bit vector that no backend can compile, so a plan built on the constructor would have looked right until the first compile. Probing set the legal widths, and separately found that no ISA has an integer vector divide.

A performance claim needs a measurement, including when it says no. Parallel sema was written, worked, and produced byte-identical output at every thread count — then measured at 1.20x against a 2.5x ceiling, because 40% of a check runs inside the type pool's exclusive critical sections. Reverted. The number is the deliverable.

A plan's stated reason is checkable. W4.5 was scheduled after W4 because exhaustiveness "wants comptime type info". Three greps and a two-line program showed it does not: the enum member set is already in the pool during checking. The wave moved forward and the table records the amendment — a wave order resting on a dependency that does not exist is a plan contradicting itself.

Check the query a claim is about. "The arithmetic around a #run is not folded" was true of the **built** MIR and false of the **optimized** one — and the corpus only ever snapshots the built query, so nothing the tests display would have shown it. A claim about optimisation needs a probe of the optimized body.

A well-typed placeholder is invisible to every gate this project has. t := Point; — a type bound to a local — type-checked cleanly and **both engines exited 0**, storing an undefined value into a slot of a type with no runtime layout at all. Three of these now, and each was found by asking a question no test asks: what does this lower to? The corpus checks exit codes and snapshots the built MIR; a construct that is legal, silent and unread sits outside all of it.

A test naming an unimplemented thing has a one-wave shelf life. The refused-body test has now had its construct replaced twice, both times because the wave after it implemented the gap it named. It uses something refused **by design** now. And one of this wave's own new tests passed vacuously until it was teeth-checked.

The language today

WORKS, END TO END

Full integer tower, bool, string, pointers
float32 and float64, plain IEEE-754, no traps
struct, nominal, one level
union, nominal, untagged — a cross-field read reinterprets
variant — a tagged union: a wrong-case read traps, switch destructures
enum and enum_flags, namespaced, bare dot-member, switch cases
Fixed arrays, views and [..]T dynamic arrays, bounds-checked; a length may name a constant
struct #soa(N) — one array per field, and e[j].x means e.x[j]
#simd [N]T — a vector at one of the six register widths; elementwise +% -% *% on integers, + - * / on floats, lane indexing, .count
Per-field #align N (a minimum, power of two up to 4096) and #place N (an exact offset, may overlap, may be unaligned)
..T variadics, including ..Any — arguments are pointers
cast and xx from context; operator overloading
Trapping arithmetic, wrapping variants, bitwise
if, else, while, for, break, continue, defer, using
switch with exhaustiveness checking over an enum; else
Multiple returns, named args, literal defaults
import, foreign, system_library, #scope_module, #expand macros that splice, #modify predicates, #bake_arguments, @note metadata
\$T and \$\$T procedures, Box(\$T) structs, \$N comptime-value parameters, #expand macros that splice, #modify predicates, #bake_arguments — W5 complete
Compile-time run at file scope or in a body, across files; type_info(), Any, #insert, #code
A type as a compile-time value: T :: Point aliases one, usable anywhere Point is
A type in a runtime position is refused — it has no representation to store
Traps name their source line and the live call chain
Bounds checks, strippable by build setting or #no_abc
context, a hidden parameter passed by pointer

ABSENT (WAVE)

pointer difference p - q, deferred
percent on floats, is_nan, math (W7)

a recursive variant; eliding the check in an arm
an explicit backing type
a length needing evaluation; array literals

a bare e[j]; using inside one
any other width; integer /; comparisons (need a mask type); swizzles

a struct-level #align; any packing form; an operand needing evaluation

a bare value coercing to Any
unary, index and call overloading
transmute; float printing

patterns, ranges, guards; a jump table
#must; a multi-result call in a return
a reader for @note: the message loop (W6)

inference through Box(\$T); a length needing arithmetic

a cross-file #run value; a Code value (declined)

a chain B :: A; comparing types; Type as an annotation

a per-frame line; inlined frames (none exist)
a per-index #no_abc; any other build setting

WORKS, END TO END

Procedures as values: call, pass, return, struct field

null, a context-typed pointer literal; malloc and free

An allocator in the context, installed and called through

Pointer offset $p + n$, $n + p$, $p - n$ — element-scaled, unchecked

push_context, a block with its own copy of the context

talloc / reset_temporary_storage — a per-context bump arena

ABSENT (WAVE)

a cross-file or foreign proc value; comparing one
cast to a pointer

the difference $p - q$; $p[n]$; ordering

hands out *u8 only; alignment; a growable region

No error-handling model yet — ADR-0008 reserves the slot and the first half exists (several return values); #must is owed its own ADR. **No GC and no RAI**, which is a design value rather than a gap. **A union is untagged**, so reading a field other than the one last written reinterprets bits: documented in three places because it cannot be diagnosed. **Indirect calls work** (ADR-0059) — this note used to call them the project's largest gap, and it went stale where nothing could catch it, which is the argument for a dashboard whose generator is committed. **The largest remaining gaps** are a compiler message loop (W6), File and a merge sort (W7), and semantic tokens (W9).

Compiler internals, 17 crates

STAGE	STATE	NOTE
Lexer, parser, CST, typed AST	works	Hand-written, error-recovering, trivia-preserving
Formatter	works	Pure function over the CST; has lost a construct in most waves that added a node kind — #simd made it 9
HIR, name resolution, modules	works	Flat import merge; cycles legal; export filtering
InternPool: types, values, layout	works	One layout computation and one integer evaluator, shared. Behind an RwLock: reads share, interning excludes
Sema: signatures, checking	works	118 codes, E0286 next free; no const-eval here, by design
MIR: typed SSA	works	Block parameters, notphis; explicit bounds check and zeroing
Mid-end	5 passes	Inline (non-leaf, bounded rounds), forwarding (cross-block), const-prop, DCE, plus the bounds-check strip. -OO skips all of it
Const-eval	works	Runs MIR through the bytecode VM
VM: register bytecode, libffi	works	No JIT; indirect calls, malloc from its own region; a vector is memory and an elementwise loop
Cranefift back end	works	Aggregate returns via sret; indirect calls via func_addr; a vector is one register
LLVM back end	works	Via inkwell + LLVM 21, behind a default-off cargo feature; gate 7 is its own test run
Language server	13 caps	Diagnostics, hover, goto, completion, rename, actions, hints, symbols, signature help; semantic tokens absent (W9)
Neovim integration	works	Runtimethat dir, no plugin manager; 166 checks
Driver	stub	Should consume the workspace notion that now exists

Roadmap: W1 through W5 and W8 closed; W6 and W7 open; W9 and W10 ahead

W1 Data	done	Numeric tower, wrapping ops, enum, enum_flags, union, arrays, views, cast, xx, operator overloading. Closed by ADR-0048.
W2 Flow and scope	done	for with it and it_index, labelled break, defer, using, multiple returns, named and default arguments, #scope_module. Closed by ADR-0054; ADR-0055 then closed a gap six corpus files had carried for eleven waves.
W3 Runtime core	done	context (ADR-0057), the bounds-check build setting (ADR-0058, finishing ADR-0003), indirect calls (ADR-0059), null plus a memory source (ADR-0060/0061), the allocator protocol (ADR-0062), push_context (ADR-0063), pointer arithmetic (ADR-0064), temporary storage (ADR-0065), and traps with backtraces (ADR-0066) — a trap names the frames that were live, byte-identically in both engines. Closed by ADR-0066; a source-level backtrace with inlined frames is deferred, because inlined frames have no runtime existence.
W4 Comptime	done	Delivered in sub-waves, because a 10-14 week wave cannot be verified the way a one-ADR wave can. All ten have shipped, and W4 is complete as scoped. A #run may call an imported procedure and appear in a body (ADR-0069), turning two internal compiler errors into working programs. An array length may name a constant (ADR-0070), which replaced the scheduled aggressive const folding after probing showed const-prop already did it. A type is a compile-time value (ADR-0071), which closed a silent miscompile — a type bound to a local compiled to an undefined value in a slot with no layout. Insert of a string literal lowers where it is written (ADR-0072), in the enclosing scope, every synthesized span pointing at the directive because jr-diag clamps an out-of-range offset rather than rejecting it. And a computed insert (ADR-0073) evaluates its operand at compile time and splices the text — the point sema and the VM become mutually recursive, PLAN section 5's named top risk, broken by an acyclic pre-pass that reuses the constant evaluator and re-lowers only the affected bodies rather than by salsa fixed-point recovery. An aggregate compile-time value (ADR-0074): a #run returning a struct or array interns as its element values rather than a byte image, because the pool is target-independent and an image is not. And type_info(T) (ADR-0075), reflection's first half: a type's kind, name, size and alignment, the numbers coming from the same layout of every real layout decision uses, so reflection cannot disagree with the layout it describes. Type_Info is declared in modules/Basic in Jairs rather than inside the compiler, because it has to be spellable — no compiler-declared type can be named at all — and the resulting dependency on a declaration the compiler does not own is validated on lookup, so editing that struct is a diagnostic rather than a read of whatever now sits at the old offset. Getting there first needed a constant that may hold a string, which ADR-0074's own closing claim said was already done and was not: the fourth false scheduled dependency this project has found, and the first where the false claim was its own ADR about the very next wave. And Any (ADR-0076, ADR-0077), reflection's second half: any_of erases a value to a {Type_Info, *u8} pair and any_as reads it back, trapping unless the type matches — the erasing pointer conversion allowed only at that boundary, because a general one would make a wrong pointee type a silent wrong read. Nothing is reinterpreted, so neither back end needed a line. The checked read needed a runtime type identity the four-field Type_Info did not have — two type_info calls have different addresses, size and alignment collide, and name is unsound because a local and an imported type share a spelling — so Type_Info gained a stable id, the pool id the compiler already uses. And Type_Info's fixed-size per-kind facts (ADR-0078): a struct's field count and an array's or pointer's element type, added as flat s64 fields because a count is a number and an element type is a pool id — so they need none of the memory-ownership decision the variable-length field list does, which stays deferred. And #code (ADR-0080), unquoted source spliced into the enclosing scope — deliberately sugar over #insert, since #insert of a named constant already worked, so what it adds is no quoting and a body parsed where it is written. A Code value is declined rather than deferred: a quoted syntax tree is worth representing only once something can inspect or transform one. The same sub-wave refused a shipped silent miscompile found while probing (ADR-0079): a pointer or view inside a compile-time aggregate interned the evaluator's own address as an integer, giving 48 in one engine and a segfault in the other with no diagnostic. Out of W4's scope, each with a recorded reason: Type_Info's variable-length field list, which needs a declared static-data mechanism and is owed its own wave; a bare value coercing to Any, which needs a materialised temporary; and a #run reading another file's constant, which now reports itself rather than an internal error.
W4.5 Pattern matching	done	switch with exhaustiveness checking, a bare dot-member as a case (settling ADR-0041 §2 step 5), and a tagged variant type (ADR-0067, ADR-0068). Reordered ahead of W4 after checking showed its stated dependency on comptime was a want rather than a need. The variant follows ADR-0045 §1's own instruction — a different declaration form, not a change to union — and union is untouched, still untagged and still one word smaller.
W5 Polymorphism	done	COMPLETE in fifteen sub-waves (ADR-0081 to ADR-0097). The last piece, #bake_arguments specialisation, lowers a declaration to a REAL procedure — a clone with the baked parameters dropped, their literals substituted and the kept ones remapped, which is the same machinery \$N instantiation uses, so W5 ends on a reuse rather than a new mechanism. A baked value must be a literal: ADR-0096 planned to use the const-eval pre-pass and building it showed that pre-pass runs AFTER lowering. #bake_arguments has its surface (ADR-0096) — a partial application producing a specialised procedure; its operand is a call so the named-argument spelling is the ordinary one, and its specialisation is refused E0276 pending the last W5 sub-wave. That refusal replaced a leaked gap report (the compiler could not lower main, please report it) which is right for an unknown gap and wrong for a named one. #modify is COMPLETE (ADR-0093/0094/0095): a compile-time predicate over an instantiation runs when a call binds the type variables, and a false refuses that instantiation (E0275) — so a template enforces its own constraints in code. It is hosted in file_mir, the only place with the expanded tree, its MIR and the VM, and rejections ride the existing diagnostics channel so it needed no new query. A predicate that fails to RUN is deliberately not a rejection. E0274 was retired when the predicate began running — the fourth by-

design refusal raised then lifted. #modify has its surface (ADR-0093): a compile-time predicate over an instantiation, guarding a template in code rather than a comment. The block parses (the one procedure attribute carrying a block) and formats with its body; a call is refused E0274 pending evaluation, because a parsed-and-ignored predicate would accept calls the author rejected — ADR-0058 rule for the third time. Its evaluation is designed and deferred. type_info(T) now reflects a BOUND type variable (ADR-0092) — a \$T procedure can ask its own bound type size, field count or identity, each instantiation seeing its own. That was found missing while designing #modify, whose predicate needs exactly it, and fixing it also turned a sixth leaked internal error into working code. The #expand SPLICE works (ADR-0091): a call splices the macro body into the caller scope, so a macro can modify the caller local — deliberately unhygienic like Jai. A generated prelude binds each argument once (substituting per use would re-evaluate a side-effecting argument), and expression position gets a generated result local so one mechanism serves both. The MIR shows no calls at all. Refused by design: an early return (E0273), a void macro in expression position, a cross-file call (E0272, which had been reaching the VM as an internal error). looks_like_proc_signature needed #expand too — the token-set trap for the fifth time, since a void macro reaches neither arrow nor brace. #expand macros have their surface (ADR-0090): a macro parses, formats and checks like any procedure, and a call is refused E0272 pending the splice — a refusal that ships WITH the surface, because without it #expand was accepted and silently ignored (a macro behaved as an ordinary procedure). jr-fmt dropped #expand on the first run, caught by gate 5. \$N comptime-value parameters are complete — surface, instantiation, and [N]T sized by one (ADR-0089), where two instantiations get genuinely different array types from one declaration. They work end to end (ADR-0087 surface, ADR-0088 build): make :: (\$N: s64) called as make(5) evaluates the argument via the same acyclic pre-pass #insert uses, and appends a concrete procedure with N baked into the body — parameter list drops the \$N, each reference to N becomes a literal. Two calls at the same value dedupe (ADR-0005 extended to values), distinct values instantiate separately. Mixed comptime+runtime params (scaled :: (\$N: s64, factor: s64)) pass only the runtime one at the call site — a per-call arg-mask filters at MIR, teeth-checked (disabling it makes the verifier catch an arty mismatch). E0271 refuses a non-constant argument at the call's span. Before that, \$T procedures work end to end (ADR-0081-0084): a \$T parameter is inferred from the call — directly or through a pointer or view — instantiated once per distinct tuple of bound types, checked per instantiation, and run as an ordinary procedure in both engines, so nothing polymorphic survives to the back end. And polymorphic structs (ADR-0085, built per ADR-0086): Box :: struct(\$T) { value: T; } used as Box(s64) is a type constructor, and Box(s64) and Box(bool) are distinct types from one declaration with substituted fields and layouts, told apart in the pool by the type argument in the key the way [2]s64 and [3]s64 are. It changed the pool's most load-bearing invariant — a struct's identity was its declaration site — and was landed in two commits, a zero-behaviour-change representation refactor proven by an unchanged snapshot and test count, then the parameterised behaviour, so a half-built type-identity change could not hide a miscompile. W5 is done; next is W6 Metaprogram then W7 Stdlib (modify, bake_arguments, expand), and the deferred struct pieces (inference through Box(\$T), using on one, cross-file, recursive List(\$T)), each a refusal today rather than a gap.

W6 Metaprogram

in progress OPEN, four sub-waves shipped, and the wave headline claim is MET: a metaprogram can find declarations by note and generate code for each one. @note attaches metadata to a declaration (ADR-0098) — @deprecated, @requires "x" — its own node kind rather than a generic attribute, because a note is DATA for a metaprogram while the directives are INSTRUCTIONS to the compiler. has_note and note_value READ it at compile time (ADR-0099), folded in sema with no VM and no new query — unlike type_info, which folds later because it needs a layout; a note answer is in the HIR the checker already holds. The first argument is the declaration itself rather than its name as text, so a misspelling is an unresolved name instead of a silent false, which is the same silence the formatter dropped notes had. An absent note answers false and empty and is NOT an error: asking whether a note is present is the point, the opposite call from any_as which traps, and the difference is that any_as would otherwise return garbage while this returns the truth. noted_count and noted_name QUERY the file without naming declarations (ADR-0100), in declaration order — the one order a reader can predict, since a name sort rennumbers every unrolled index when a declaration is inserted and a hash order makes one program answer differently between runs. And noted_insert GENERATES (ADR-0101): one template emitted once per noted declaration, hash standing for each name, spliced by #insert — so a single line generates a call to every noted procedure. The loop lives inside the FOLD, and that corrects ADR-0100 scope the way ADR-0094 corrected ADR-0093: folding cannot take a for variable, but that forbids a loop in the PROGRAM and says nothing about a loop inside the FOLD — and for generation the fold is the only thing that can work at all, since generated code must exist before checking, so a run-time loop could not declare a procedure or emit a statement. Probing found that #insert note_value(f, "gen") ALREADY worked, a note payload spliced as code, shipped and undocumented. Two refusals were leaked errors until this wave probed them: == on two strings reached the VM as expected a scalar found an aggregate (E0278), now refused for every aggregate the way a view equality already was, by a structural test since size and alignment cannot tell an s64 from a two-field struct of s32s. And a latent miscompile was fixed (ADR-0101): a folded value keyed by ExprId is STALE once a body expands, because a computed #insert rennumbers every id after its splice — so with two computed inserts the second value landed on a different expression, putting a string on an arithmetic operand, and it surfaced as a VERIFIER PANIC rather than a diagnostic. That is the sharpest well-typed placeholder this project has had: the two earlier ones were placeholders that happened to be legal values, while this is a genuine value from the same program merely attached to the wrong expression, so nothing in the type system can see it is wrong. Remaining in W6: run-time INSPECTION — a loop reading declarations as values — which needs a compiler-emitted static-data table and lifts Type_Info variable-length field list at the same time, plus #run build() build scripts, plugin hooks and workspaces.

W7 Stdlib

in progress OPEN, seventeen sub-waves, and every one was driven by a refusal or a bug rather than a checklist. String (ADR-0103): equal, compare, starts_with, ends_with, find, contains, byte_at, is_empty, NONE of which allocate — it exists because ADR-0099 refused == on two strings (same storage and same contents are both plausible for a {data, count} pair) and named a byte loop as the fix, which is a library job rather than an operator since an == that looped would be the only implicitly-looping operator in the language. Its OWN module rather than more of Basic, and the deciding argument is not size: adding to Basic would mean nothing ever tested that TWO modules can be imported at once. Nothing allocates deliberately — the mechanism exists but the CHOICE between context-allocator, explicit parameter and always-temporary does not, and settling it in passing is how a library gets an accidental convention. Sort (ADR-0104) is the first POLYMORPHIC library code; the CALLER supplies the comparison because resolving an operator inside a \$T template against the instantiated type is a lookup instantiation does not do, and insertion sort is chosen for STABILITY and NO ALLOCATION rather than speed. Writing it found TWO leaked internal errors: an imported procedure used as a VALUE (representable all along, a three-line bridge missing) and a call to an imported TEMPLATE (now E0268, with a diagnostic that NAMES THE WORKAROUND and a corpus file proving the workaround works) — both hiding behind a stale comment that said something checkable nobody had checked. Array (ADR-0105) is FIXED-CAPACITY, and three PROBED refusals decided that rather than effort: a malloc'd region cannot be typed, inference through a parameterised struct is deferred, and such a struct CANNOT CROSS A MODULE BOUNDARY — the last found by importing the module,

which makes a polymorphic struct in a module unusable by everyone. Routing around them with hand-computed byte offsets was rejected: the standard library is the worst place to route around a deliberate refusal. Typed allocation (ADR-0106) is a LANGUAGE change: `size_of`, typed and untyped make heap storage reachable while `cast` stays refused — typed is not SAFER but VISIBLE, its target type a type argument at a searchable boundary. The plan was amended mid-build because MIR cannot reach `malloc`, so the library allocates and only the retyping is an intrinsic. It fixed a pre-existing store-to-load forwarding miscompile (forwarding deleted the very store-then-load that performs a retype), and the first fix was too broad — the optimized-MIR snapshot caught the lost optimisation, which is exactly the job a snapshot has since an optimisation quietly not happening is invisible to every other gate. List (ADR-0107) is the genuinely growable array, doubling from four so `n` pushes cost $O(n)$ amortised; a NEW module rather than a rewrite because contracts differ — an `Int_List` OWNS memory and there are no destructors. It produced the corpus differential's FIRST REAL CATCH: 247 in the VM against 255 natively, because the VM satisfied `malloc` from the FRAME BUMP CURSOR, so heap memory allocated in a callee was reclaimed on return and read back as ZERO rather than garbage (release zeroes for determinism, making the symptom a clean wrong answer). Every earlier catch was a construct BOTH engines got wrong; this was one right and one wrong, which is what two implementations exist to expose. ADR-0108 reports every reachable file's diagnostics: a root whose imported module was broken used to check clean and fail inside an engine, because `file_diagnostics` answers for ONE file — resolution was never wrong, nothing ASKED the module. It rejects programs the compiler used to accept, every one of which was going to fail anyway later and less comprehensibly. ADR-0109 adds `view(p, n)`, which builds a `[[T` from a pointer and a count so `sort_ints(elements("l"))` sorts a growable list IN PLACE — the library composes — and it revisited ADR-0044 whose stated reason for refusing `view.data` had expired (it wanted pointer arithmetic that now exists); the answer was the missing constructor, not the exposed field. And ADR-0110 makes a call through a NULL procedure pointer trap: found while probing what allocator String should use, since `context.allocator` is null until installed, and both engines were wrong differently — the VM decoded null to file 0 proc 0, an arbitrary real procedure, giving a message about an arity nobody wrote, while native would have jumped to address zero. The VM handle is now biased by one so zero means null, which valid/048 proved necessary by calling the first procedure in the file. Seven leaked internal errors have now become real diagnostics or working code. Then String grew its allocating half (ADR-0111: `concat`, `substring`, `to_upper`, `to_lower` through `context.allocator`, caller frees), Math shipped the exact closed-form functions (ADR-0112) and Random a deterministic `xorshift64` generator (ADR-0113, which surfaced a language gap: a u64-range constant has no name-colon-type-colon-value form). ADR-0114 then let a FLOAT cross the FFI boundary, passed in a float register rather than a word in both engines — the unblocker Math named — and ADR-0115 collected it, adding Math's transcendentials (`sqr`, `sin`, `cos`, `exp`, `ln`, `powf`) as `libm` wraps bit-identical in both engines because both call the same `libm`. Eight modules now: Basic, String, Sort, Array, List, Math, Random, Map. ADR-0116 added `Int_Map`, an open-addressed hash table, and writing it caught the SECOND engine divergence: the wrapping operators decoded to i128 and overflowed for two large u64 values, panicking the compile evaluator while native code was right. Both differential catches have been in arithmetic or memory the native path did in hardware while the VM modelled it in Rust. Then the biggest language unblocker the wave had left: ADR-0117 lets a PARAMETERISED STRUCT CROSS A MODULE BOUNDARY, which three library sub-waves had named. It was not a lookup change — a parameterised struct fields are resolved per instance under the caller arguments, and its own file cannot do that, so the IMPORTER resolves them, which needs the field type tree in the DECLARING file arena. The check phase now receives the imported HIR. Identity stays the declaring file, so `Box(s64)` is one type across importers, and the pool needed nothing: ADR-0086 instance-keyed field map already covered it. ADR-0118 collected in the modules — `Array($T)` and `List($T)` declare their storage ONCE — and ADR-0119 finished the job by letting an intrinsic take a parameterised type argument (three separate refusals of one construct: `sema`, the resolver type-position flag, and MIR scan), which unblocked `Map($K, $V)`. All three containers are generic structs now, and the MIR snapshots did not move, which is the right outcome since the instances lay out as the concrete structs did. Remaining: a merge sort and binary search, File and the OS modules, JSON, Compiler — plus three named unblockers all in PROCEDURE polymorphism: inference through a parameterised struct, cross-file `$T` instantiation, and using on an imported struct.

W8 Performance

done DONE in eight sub-waves (ADR-0142 to ADR-0149). An optimisation level -O0/-O1 whose real deliverable is the check the mid-end never had: every corpus program behaves identically at both levels, so a wrong answer is attributable to lowering rather than a pass. The LLVM back end via `inkwell` behind a default-off feature, making the differential three-way — it agreed with the VM on all 114 executable programs on the FIRST run, because both native engines read the same MIR and ask `jr-pool` the same layout questions. `#align` and `#place`, the first features whose whole implementation is a layout FEATURE rather than a fix. `Inline` maturity: the leaf rule is gone, termination is a bounded round count, and forwarding follows a single-predecessor chain — 024-hello's optimised MIR now flattens three call layers the old pipeline could not. A published compile-throughput number, and `heap_sort` chosen by a COMPARISON COUNT rather than a wall clock, because this project can measure its own throughput and deliberately cannot measure the programs it compiles. `#soa`, where the sugar IS the feature — without `e[i].x` it buys nothing over writing `[N]T` by hand. `#simd`, whose legal widths were set by probing `Cranelift` rather than by taste, and whose integer lanes take the wrapping operators because no vector add can trap. And parallel `sema`, MEASURED AND REFUSED: 1.20x against a 2.5x ceiling, reverted, with the 40%-serial measurement and two named blockers left behind.

W9 Tooling depth

not started Semantic tokens are the one LSP capability still missing; the other thirteen landed early. Richer DWARF (locals, struct layouts) for `lldb`, and `Neovim` packaging — VS Code was descoped by ADR-0036, and any LSP client works unpackaged.

W10 Graphics, in Jairs

not started The last wave the plan reaches. Window creation through foreign calls (`Cocoa`), a GPU layer (`Metal` then `Vulkan`), an immediate-mode 2D renderer, image decode, immediate-mode UI, audio. ALL library work written in Jairs, no compiler changes — which is why it is gated on W5 (complete) and W7 (open), and why it is the wave that tests whether the language is actually usable.

What the last stretch did, and the two claims it settled

W8 SHIPPED, IN EIGHT SUB-WAVES

ADR-0142 through 0149. Test count 986 to **1033** (1034 with LLVM compiled in), corpus 210 to **237** files, ADRs 120 to **149**. The wave's shape is worth more than its list: seven sub-waves shipped a feature and the eighth shipped a **number**, which is what closing a performance wave honestly looks like.

A third engine costs less than it looks when the seam was drawn early. ADR-0009 put every `cranelift-*` reference behind `jr-codegen::Backend` twelve waves before there was a second back end to justify it. Paying that in ADR-0143 took one crate and found exactly what such a seam is for:

TWO CLAIMS THE MEASUREMENTS SETTLED

A vector type's legal widths are a machine fact, not a design choice.

`Cranelift's Type::by` cheerfully constructs a 256-bit vector, reports its width, and then fails to compile a single `iadd` on it. A design that trusted the constructor would have looked complete until the first build. Probing set the six legal shapes, and separately established that no ISA has an integer vector divide — so `/` on an integer vector is refused rather than quietly scalarised into something slower than the scalar code it replaced.

Parallelism was refused by its own numbers. A parallel `jr` check was written, worked, and produced byte-identical output at 1, 2, 4, 8 and 12

TrapKind and the trap helper moved into the shared crate, because they are the **words** trapping programs print, and a second copy would have been a second chance to drift from the bytes the differential compares.

Two features needed no engine change at all, and both for the same reason. #soa transforms field types in the type checker **before** layout runs, and a #simd vector has the array's exact layout — so jr-pool's single layout computation carried both into the VM and both back ends. #simd needed no MIR lowering for lane access whatsoever, which is the strongest evidence its “same layout, different operations” reading is right.

threads. Instrumenting the pool guard found 571 acquisitions holding it for 30 ms of a 74 ms check — 40% serial, so Amdahl caps any driver-level parallelism at 2.5x, and the measured process-level gain was 1.20x on a clean tree and 1.01x on a mixed one. It was reverted: 1.2x does not pay for a deadlock mode that appears **only** under threads. The parallel-codegen probe was worse — it looked like 84% lock contention and was actually measuring duplicated work, because no program here has more than four files.

SETTLED SINCE THE LAST DASHBOARD

Everything is merged. This box used to say main sat at ec150a5 with twenty-three waves stacked on branches ahead of it. main is now at 8e2dafe, and every wave through W8 is merged with --no-ff, one merge commit per wave — so git log --merges main reads as the wave history. The branch names were corrected first: two W8 sub-waves had been committed onto a sibling's branch, and feat/simd and feat/parallel-sema now name their own work.

The one open slice criterion is still a verified Linux x86-64 CI run — configured, never run, which makes it a decision rather than a technical gap. Two other things are owed and cheap: the security audit's remaining two dispatches (by hand — six subagent dispatches returned empty), and tree-sitter test added to gate 6, which today catches grammar drift but not a broken rule.

Sources: PLAN.md §1.5 and §7, the ADR directory, docs/decisions/DECISIONS.md, and all seven gates run today on main. Every number was measured rather than carried forward — the test count from a full workspace run, the corpus count from a file walk, the diagnostic codes from the const E0nnn definitions across the crates, and the editor-check count from a real verify.lua run. Rebuild with typst compile jairs-dashboard.typ.